

Yayınla–Abone Ol Tabanlı Dağıtık Sistemlerin Analizine Yönelik Çizge Modeli ile Görselleştirilmesi

Visualization of Distributed Publish-Subscribe Systems Using Graph Models for Analysis

Onurcan Erşen¹, Mustafa Can Çalışkan^{1,2}, İbrahim Onuralp Yiğit^{1,2}, Feza Buzluca²

¹ HAVELSAN, Komuta Kontrol & Savunma Teknolojileri, 34903, Pendik, İstanbul, Türkiye

² İstanbul Teknik Üniversitesi, Bilgisayar Mühendisliği Bölümü, 34469, Maslak, İstanbul, Türkiye

Özet

Yayınla–abone ol (publish-subscribe) mimarisi modern dağıtık sistemlerin temel taşlarından biri haline gelmiştir. Bu çalışmada, yayınla–abone ol tabanlı dağıtık sistemlerin çizge modeli ile temsili ve etkileşimli görselleştirilmesine yönelik kapsamlı bir web platformu sunulmaktadır. Önerilen yaklaşım, sistem topolojisini ağırlıklı yönlü çizge yapısına dönüştürerek uygulamalar, araçlar (broker), konular (topic), kütüphaneler (library) ve altyapı düğümleri arasındaki karmaşık bağımlılık ilişkilerini görselleştirmektedir. Çizge modeli üzerinde PageRank, arasındalık merkeziliği ve eklemleme noktası analizi gibi topolojik metrikler hesaplanmakta, kutu grafiği (box plot) tabanlı istatistiksel sınıflandırma ile bileşenlerin kritiklik düzeyleri belirlenmektedir. Platform, Next.js/React tabanlı modern web arayüzü, FastAPI backend servisleri ve Neo4j çizge veritabanı entegrasyonu ile kurumsal ölçekte kullanılabilir bir çözüm sunmaktadır. Kullanıcılar etkileşimli ağ grafikleri (2 ve 3 boyutlu görünüm), istatistiksel grafikler ve kapsamlı analiz sayfaları üzerinden veri görselleştirmelerine erişebilmektedir. Deneysel sonuçlar, önerilen yaklaşımın sistemin güvenilirliğine, bakım kolaylığına, erişilebilirliğine ve kırılganlığına etki eden kritik bileşenlerin tespitinde güvenilir sonuçlar ürettiğini göstermektedir.

Anahtar Kelimeler: Yayınla–abone ol mimarisi, çizge modelleme, web tabanlı görselleştirme, dağıtık sistemler, kritik bileşen analizi, kutu grafiği sınıflandırma

Abstract

Publish-subscribe architecture has become a cornerstone of modern distributed systems. This study presents a comprehensive web platform for representing and interactively visualizing publish-subscribe-based distributed systems using graph models. The proposed approach transforms the system topology into a weighted, directed graph that illustrates complex dependency relationships among applications, brokers, topics, libraries, and infrastructure nodes. The criticality levels of components are determined by computing topological metrics, such as PageRank, betweenness centrality, and articulation point analysis, complemented by BoxPlot-based statistical classification. The platform provides an enterprise-grade solution with a Next.js/React-based modern web interface, FastAPI backend services, and integration with the Neo4j graph database. Users can access interactive network visualizations (2D and 3D views), statistical charts, and comprehensive analytical dashboards. Experimental results demonstrate that the proposed approach reliably identifies critical components across the system's reliability, maintainability, availability, and vulnerability.

Keywords: Publish-subscribe architecture, graph modeling, web-based visualization, distributed systems, critical component analysis, BoxPlot classification

1. Giriş

Dağıtık sistemler, bulut bilişim, nesnelerin interneti (IoT) ve otonom sistemler gibi modern teknolojilerin temelini oluşturmaktadır. Bu sistemlerin karmaşıklığı arttıkça, bileşenler arası iletişimin yönetimi ve sistem mimarisinin anlaşılması giderek zorlaşmaktadır. Yayınla–abone ol (publish-subscribe) mimarisi, gevşek bağlı (loosely coupled) ve ölçeklenebilir sistemlerin tasarımı için yaygın olarak tercih edilen bir iletişim paradigmasıdır [1].

Yayınla–abone ol sistemlerinde, yayıncılar (publisher) mesajları belirli konulara (topic) gönderirken, aboneler (subscriber) ilgilendikleri konulara abone olarak mesajları alırlar. Bu dolaylı iletişim modeli, sistem esnekliğini artırırken bileşenler arası bağımlılıkların izlenmesini zorlaştırmaktadır. Özellikle onlarca aracı (broker), yüzlerce uygulama ve binlerce konu içeren kurumsal ölçekli sistemlerde, mimari karmaşıklık kritik bir sorun haline gelmektedir. Yiğit ve Buzluca [2] yaptıkları çalışmada dağıtık sistemlerin çizge tabanlı olarak modellenmesi ve bu sistemlerde kritik bileşenlerin

belirlenmesi için kapsamlı bir metodoloji sunmuşlardır. Bu çalışmanın temel motivasyonu, Yiğit ve Buzluca'nın önerdiği çizge tabanlı bağımlılık analizi metodolojisini kullanıcı dostu bir web platformu aracılığıyla erişilebilir kılmaktır. Önerilen platform, sistem topolojisinin etkileşimli görselleştirilmesini sağlayarak mimari karmaşıklığın anlaşılmasını kolaylaştırmaktadır. Statik analiz yaklaşımı sayesinde çalışma zamanı izleme maliyetleri ortadan kaldırılırken, sistemin kalitesi açısından kritik bileşenlerin güvenilir bir şekilde tespit edilmesi mümkün olmaktadır.

Çalışmanın temel katkıları şu şekilde özetlenebilir: (i) yayımla-abone ol mimarisine sahip dağıtık sistemlerin çizge modeli web tabanlı görselleştirmesi gerçekleştirilmiştir. (ii) 2D ve 3D etkileşimli ağ grafikleri ile sezgisel kullanıcı deneyimi sağlanmıştır. (iii) kutu grafiği (box plot) tabanlı istatistiksel sınıflandırma ile kritiklik analizi iyileştirilmiştir. (iv) Kurumsal ölçekte kullanılabilir modüler bir platform mimarisi tasarlanmıştır.

Makalenin geri kalanı şu şekilde organize edilmiştir: İkinci bölümde ilgili çalışmalar incelenmektedir. Üçüncü bölümde çizge modeli özetlenmektedir. Dördüncü bölümde görselleştirme yaklaşımı açıklanmaktadır. Beşinci bölümde deneysel değerlendirme sunulmaktadır. Altıncı bölümde sonuçlar ve gelecek çalışmalar tartışılmaktadır.

2. İlgili Çalışmalar

2.1. Dağıtık Sistem Görselleştirmesi

Dağıtık sistemlerin görselleştirilmesi, yazılım mühendisliği alanında uzun süredir araştırılan bir konudur. Sambasivan ve diğerleri [3] dağıtık izleme sistemlerinin görselleştirmesi için çeşitli yaklaşımları incelemiştir. Bu çalışmalar genellikle çalışma zamanı verilerine dayanmakta ve yüksek izleme maliyetleri gerektirmektedir. Jaeger [18] ve Zipkin [17] gibi dağıtık izleme araçları bu kategoride ön plana çıkmakla birlikte, devreye alma öncesi analiz çalışmaları için elverişli nitelikte değildir.

Çizge tabanlı görselleştirme araçları arasında Gephi [4] ve Graphviz [5] öne çıkmaktadır. Ancak bu araçlar genel amaçlı olup, yayımla-abone ol sistemlerinin özgün yapısını yansıtmamaktadır. Modern web tabanlı görselleştirme için WebGL tabanlı kütüphaneler [11] yaygın olarak kullanılmaktadır.

2.2. Yazılım Bağımlılık Analizi

Yazılım bağımlılık analizi, sistemin yapısal özelliklerini anlamak için önemli bir araştırma alanıdır. Zimmermann ve Nagappan [6] bağımlılık ağlarının yazılım kalitesi üzerindeki etkisini incelemiştir. Cai ve diğerleri [7] mimari bağımlılık analizi için DRSpaces aracını geliştirmiştir. Çizge veritabanları [12] ve ağ analiz kütüphaneleri [13] bu alanda önemli altyapı bileşenleri olarak kullanılmaktadır.

2.3. Çizge Tabanlı Kritiklik Analizi

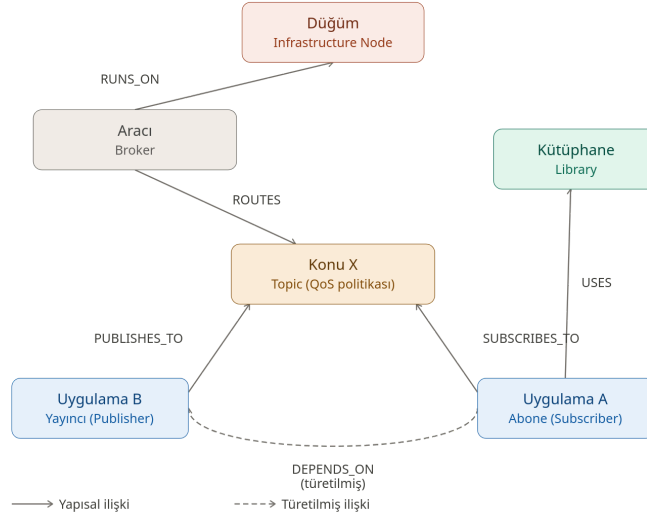
Çizge merkezilik metrikleri, ağ yapılarında önemli düğümlerin belirlenmesi için yaygın olarak kullanılmaktadır. PageRank [8], arasındalık merkeziliği [9] ve HITS algoritması [10] sosyal ağ analizi ve web madenciliğinde başarıyla uygulanmıştır. Yiğit ve Buzluca [2], bu metriklerin yayımla-abone ol sistemlerinin kritiklik analizine uyarlanmasını ve çok katmanlı çizge modeli ile bütünleştirilmesini gerçekleştirmiştir. Bu çalışma, Yiğit ve Buzluca'nın sunduğu çizge tabanlı modelleme ve analiz yaklaşımının görselleştirme boyutunu genişletmektedir.

3. Çizge Modeli

Yayımla-abone ol tabanlı dağıtık çalışan bir sistem, Yiğit ve Buzluca'nın [2] önerdiği yaklaşımda yönlü ağırlıklı çizge $G = (V, E, \tau, w)$ olarak modellenmektedir. Bu modelde V sistem bileşenlerini temsil eden düğümler kümesi, $E \subseteq V \times V$ bileşenler arası ilişkileri temsil eden kenarlar kümesi, $\tau: V \cup E \rightarrow T$ düğüm ve kenar türlerini belirleyen tür fonksiyonu ve $w: V \cup E \rightarrow \mathbb{R}^+$ ağırlık fonksiyonudur.

Çizge modeli, beş temel bileşen türünü desteklemektedir [2]: Düğüm (Infrastructure Node) altyapı bileşenlerini, Aracı (Broker) mesaj yönlendirme ara katman yazılımını, Konu (Topic) QoS politikalarına sahip mesaj kanallarını, Uygulama (Application) servis bileşenlerini ve Kütüphane (Library) paylaşılan kod bağımlılıklarını temsil eder.

Yapısal ilişkiler (RUNS_ON, ROUTES, PUBLISHES_TO, SUBSCRIBES_TO, USES, CONNECTS_TO) doğrudan girdi verilerinden elde edilmekte, türetilmiş bağımlılık ilişkileri ise kurallar doğrultusunda otomatik olarak hesaplanmaktadır. Şekil 1, önerilen çizge modelini örneklemektedir. Örnek çizge modeli, beş temel bileşen türünü ve aralarındaki yapısal ilişkileri göstermektedir. Kesikli ok ile gösterilen DEPENDS_ON ilişkisi, Uygulama A'nın Konu X'e abone olması ve Uygulama B'nin aynı konuya yayın yapması nedeniyle türetilen bağımlılığı temsil etmektedir.



Şekil 1. Örnek çizge modeli.

Çizge modeli üzerinde aşağıdaki merkezilik metrikleri hesaplanmaktadır:

- **PageRank** metriği, çizge yapısı üzerindeki bir bileşenin sistem genelindeki önem derecesini nicelleştirmekte; yüksek skora sahip bileşenler, çok sayıda kritik öge tarafından bağımlılık olarak referans verilmektedir.
- **Arasındalık merkeziliği** (Betweenness Centrality, BC) metriği, bileşenin iletişim yolları üzerindeki köprüleme potansiyelini ölçmekte olup, düğüm çiftleri arasındaki en kısa yolların bileşen üzerinden geçme oranı olarak hesaplanmaktadır.
- **Giriş derecesi** (DG_in), bir bileşene yönelen bağımlılık ilişkilerinin toplamını; **çıkış derecesi** (DG_out) ise bileşenin işlevselliği için gereksinim duyduğu dış bağımlılıkların sayısını ifade etmektedir.
- **Kümeleme katsayısı** (Clustering Coefficient, CC), bir bileşenin komşuluk ilişkilerinin kendi aralarındaki bağlantısalılık yoğunluğunu göstermektedir. Yüksek katsayı değeri, bileşenin sıkı bağlı bir alt sistemin parçası olduğuna ve bakım faaliyetlerinin etkisinin yerel kalacağına işaret etmektedir.
- **Eklemlenme noktası** (Articulation Point, AP) analizi ile sistemden ayrıldığında çizge bütünlüğünü bozarak yapıyı bağlantısız hale getiren düğümler belirlenmekte; bu durum $AP(v) \in \{0, 1\}$ ikili değişkeni ile kodlanmaktadır.
- **Tek Arıza Noktası Skoru** (Single Point of Failure, SPOF), bileşenin doğrudan bir eklemlenme noktası teşkil etmese dahi sistemin iletişim yolları üzerinde yarattığı kritik bağımlılık yoğunluğunu ölçmektedir. En kısa yolların v düğümü üzerinden geçme oranının normalleştirilmesiyle hesaplanan bu skor, AP ile bütünleştirilerek erişilebilirlik boyutunu etkilemektedir.
- **Bağımlılık yoğunluğu** (Depth Density, DD), bileşene ait çıkan bağımlılık kenarlarının ağırlıklı toplamının teorik olarak mümkün olan azami bağımlılık sayısına oranıdır. Yüksek değerler, bileşenin geniş bir küme ile kurduğu doğrudan bağımlılık nedeniyle olası arızalarda yayılım riskinin arttığını göstermektedir.

Hesaplanan topolojik metrikler, R-M-A-V (Reliability-Maintainability-Availability-Vulnerability) kalite çerçevesi kullanılarak dört ana boyuta indirgenmektedir. Söz konusu boyutların matematiksel modelleri aşağıda sunulmuştur:

$$R(v) = 0,60 \times \text{PageRank}(v) + 0,40 \times \text{DG_in}(v) \quad (1)$$

$$M(v) = 0,70 \times \text{BC}(v) + 0,30 \times (1 - \text{CC}(v)) \quad (2)$$

$$A(v) = 0,70 \times \text{AP}(v) + 0,30 \times \text{SPOF}(v) \quad (3)$$

$$V(v) = 0,55 \times \text{DG_out}(v) + 0,45 \times \text{DD}(v) \quad (4)$$

Güvenilirlik boyutu $R(v)$, bir bileşenin sistem genelindeki kritik önem derecesini (PageRank) ve maruz kaldığı doğrudan bağımlılık yükünü (DG_in) bütünleştirerek, olası arıza senaryolarında üst akış etkisini nicelleştirmektedir. Bakım yapılabilirlik boyutu $M(v)$, yüksek arasındalık merkeziliğinin müdahale karmaşıklığını artırdığı ve düşük kümeleme katsayısının komşuluk ilişkileri üzerindeki bağlantısallık yoğunluğunu zayıflattığı varsayımıyla modellenmektedir. Erişilebilirlik boyutu $A(v)$, sistem bütünlüğünü bozabilecek eklemlenme noktalarını önceliklendirmekte ve bu noktaların yarı-tek arıza noktası (QSPOF) riski ile ağırlıklandırılmasını öngörmektedir. Kırılganlık boyutu $V(v)$ ise, bileşenin işlevselliği için gereksinim duyduğu dış bağımlılıkların hacmini ve yapısal yoğunluğunu ölçümlemektedir.

Söz konusu dört kalite boyutu, Analitik Hiyerarşi Süreci (AHP) [16] yöntemiyle ağırlıklandırılarak, bileşik kalite puanı $K(v)$ aşağıdaki formülle hesaplanmaktadır:

$$K(v) = 0,43 \cdot A(v) + 0,24 \cdot R(v) + 0,17 \cdot M(v) + 0,16 \cdot V(v) \quad (5)$$

Bileşenlerin kritiklik düzeyleri, sabit eşik değerleri yerine kutu grafiği istatistikleri kullanılarak belirlenmektedir. Bu yaklaşım, veri dağılımına uyarlanabilir sınıflandırma sağlamaktadır. $Q1$ (birinci çeyreklik), $Q3$ (üçüncü çeyreklik) ve IQR (çeyrekler arası açıklık) değerleri hesaplanarak beş kritiklik düzeyi tanımlanmaktadır:

- KRİTİK ($K(v) > Q3 + 1,5 \times IQR$),
- YÜKSEK ($K(v) > Q3$),
- ORTA ($K(v) > Q2$ (medyan)),
- DÜŞÜK ($K(v) > Q1$),
- MİNİMAL ($K(v) \leq Q1$).

Bu istatistiksel yaklaşım, farklı ölçeklerdeki sistemlerde tutarlı sınıflandırma sağlamaktadır.

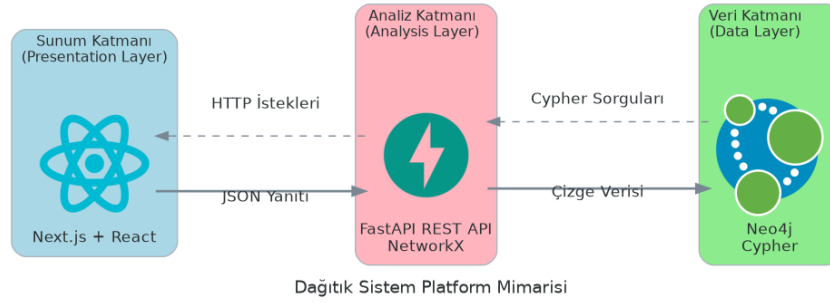
4. Görselleştirme Yaklaşımı

4.1. Platform Mimarisi

Görselleştirme platformu [2]'de tanımlanan analiz metodolojisini web tabanlı kullanıcı arayüzü ile bütünleştiren katmanlı bir mimari sunmaktadır. Şekil 2'de gösterilen üç katmanlı mimari aşağıdaki katmanlardan oluşmaktadır:

- Sunum Katmanı (Next.js + React),
- Analiz Katmanı (FastAPI REST API + NetworkX)
- Veri Katmanı (Neo4j + Cypher)

Veri Katmanı'nda Neo4j [12] çizge veritabanı üzerinde çizge modeli saklanmakta ve Cypher sorgu dili ile erişilmektedir. Analiz Katmanı'nda NetworkX [13] kütüphanesi kullanılarak çizge modeline ilişkin topolojik metrikler hesaplanmakta ve FastAPI tabanlı RESTful API üzerinden servis edilmektedir. Sunum Katmanı'nda Next.js ve React kullanılarak sunucu taraflı işleme destekli, duyarlı (responsive) web arayüzü sunulmaktadır. Katmanlar arası HTTP istekleri ve Cypher sorguları ile veri akışı sağlanmaktadır.

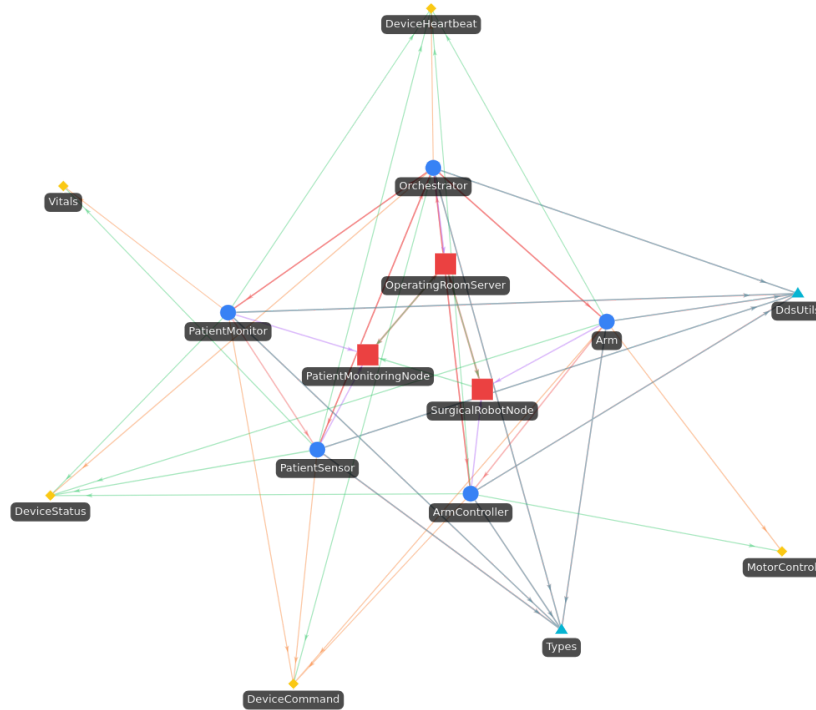


Şekil 2. Dağıtık sistem platform mimarisi.

4.2. Etkileşimli Ağ Grafikleri

Ağ görselleştirmesi için react-force-graph-2d ve react-force-graph-3d kütüphaneleri [11] kullanılmaktadır. Bu kütüphaneler, WebGL tabanlı yüksek performanslı 2D ve 3D kuvvet yönlendirmeli çizgeler sağlamakta ve büyük ölçekli grafiklerin akıcı bir şekilde işlenmesini mümkün kılmaktadır. Kullanıcılar klavye kısayolu ile 2D ve 3D görünüm arasında dinamik olarak geçiş yapabilmektedir.

Düğüm, bileşen türlerine göre renk kodlaması ile gösterilmektedir: Uygulama mavi, Altyapı Bileşeni kırmızı, Aracı gri, Konu sarı ve Kütüphane camgöbeği renkleri ile temsil edilmektedir. Kenarlar ilişki türlerine göre farklı renklerde gösterilmektedir. Şekil 3, önerilen platformun etkileşimli ağ grafiğinin örnek bir görünümünü sunmaktadır. Grafikteki düğümler ve kenarlar, sistem bileşenlerini ve aralarındaki bağımlılık ilişkilerini görselleştirmektedir. Kullanıcı etkileşimleri kapsamlıdır: fare ile üzerine gelme, tıklama, sürükleme, zoom, pan, düğüm sabitleme ve çift tıklama işlemleri desteklenmektedir.

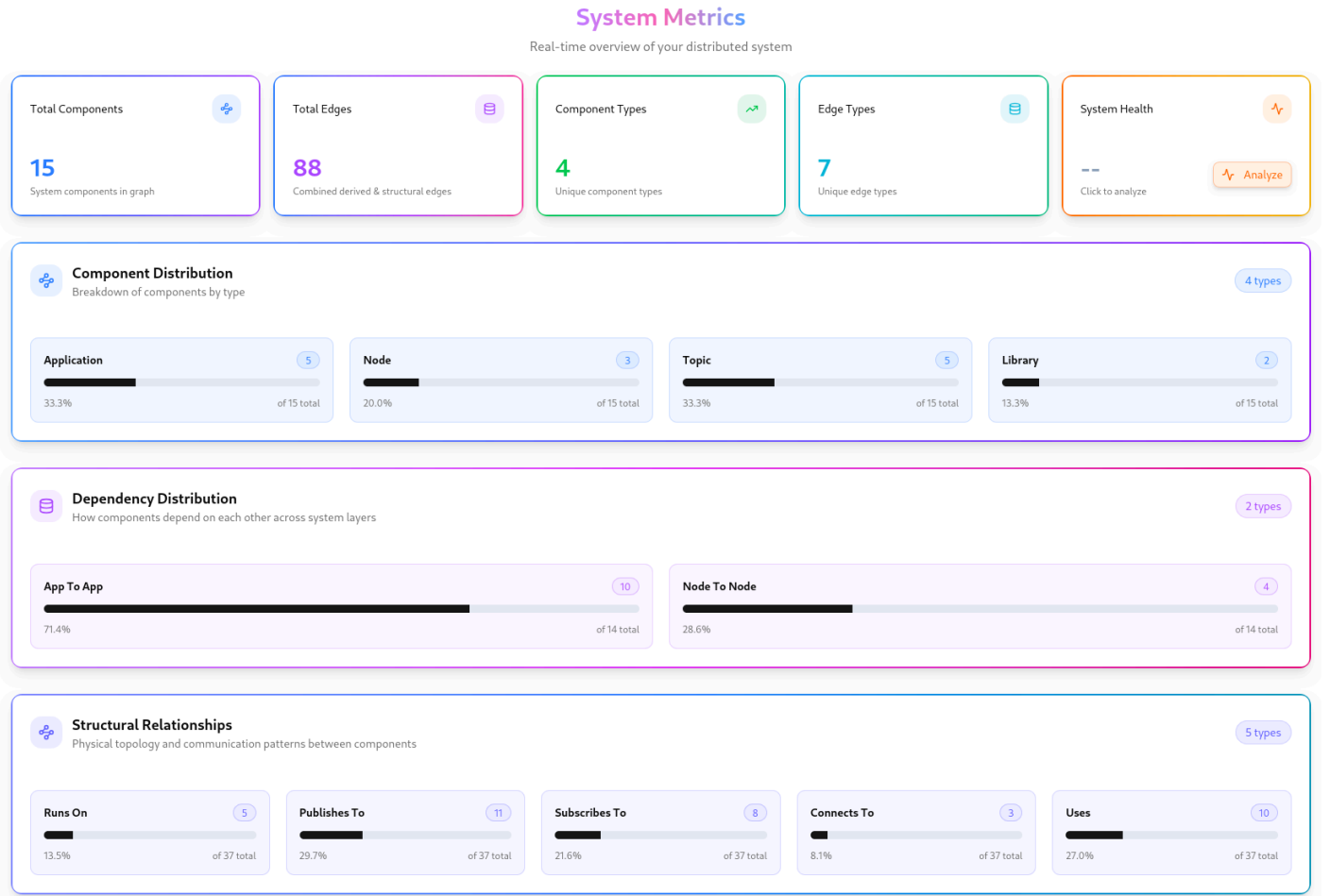


Şekil 3. Dağıtık sistem çizge modelinin 2D kuvvet yönlendirmeli çizge ile görselleştirmesi.

4.3. Gösterge Paneli Bileşenleri

Web platformu, altı ana sayfa üzerinden kapsamlı analiz ve görselleştirme sunmaktadır: Dashboard Sayfası sistem metrikleri kartları ve dağılım grafikleri, Graph Sayfası 2D/3D görselleştirme ve filtre kontrolleri, Analysis Sayfası [2]'deki kritiklik analizi tabloları ve radar grafikleri, Statistics Sayfası kutu grafiği analizi ve korelasyon matrisleri, Simulation Sayfası arıza simülasyonu ve etki analizi, Validation Sayfası Spearman korelasyonu ve F1-skor metrikleri sunmaktadır.

Şekil 4, Gösterge Paneli Sayfası'nın gerçek zamanlı sistem metriklerini gösteren kullanıcı arayüzünü sunmaktadır. Gösterge panelinde toplam bileşen sayısı, kenar sayısı, bileşen türleri ve kenar türleri gibi temel sistem metrikleri kart bileşenleri ile görselleştirilmektedir. Ayrıca, bileşen dağılımı ve bağımlılık dağılımı grafikleri, sistemin yapısal özelliklerini özetleyen istatistiksel dağılımlar sunmaktadır. Yapısal ilişkiler bölümünde, farklı ilişki türlerinin dağılımı yüzdelik oranlarla gösterilmektedir. Bu gösterge paneli, sistem yöneticilerine ve geliştiricilere mimari karmaşıklığın hızlı bir şekilde değerlendirilmesi için sezgisel bir arayüz sağlamaktadır.

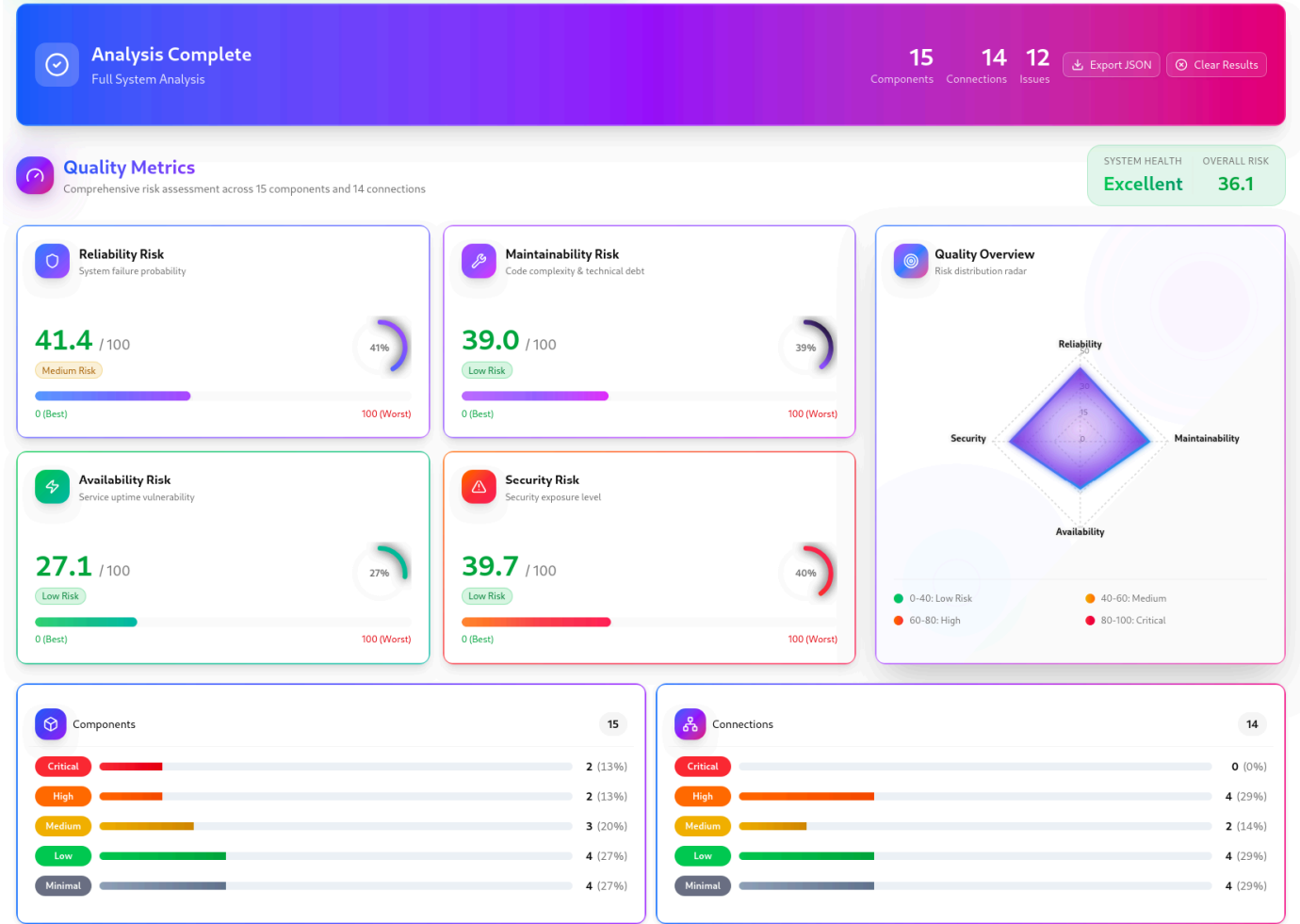


Şekil 4. Gösterge Paneli sayfası kullanıcı arayüzü.

Şekil 4'teki üst bölümde sistem metrikleri kartlar halinde gösterilmektedir. Alt bölümlerde bileşen dağılımı, bağımlılık dağılımı ve yapısal ilişkiler yüzdelik oranlarla görselleştirilmektedir.

4.4. Çok Katmanlı Analiz

Sistem, beş farklı katmanda analiz olanağı sağlamaktadır: Uygulamalar arasındaki bağımlılık ilişkilerini gösteren uygulama katmanı, dağıtık sistemdeki düğümler arasında bağımlılık ilişkilerini temsil eden altyapı katmanı, mesaj dağıtıcıları ile uygulama ve düğümler arası ilişkilerin gösterildiği arakatman katmanı ve tüm katmanların birleşimini gösteren sistem katmanı. Her katman için ayrı görselleştirme ve metrikler hesaplanmakta, katmanlar arası karşılaştırma imkânı sağlanmaktadır. Şekil 5, analiz sayfasında kalite metrikleri ve kritiklik analizi sonuçlarının sunulduğu kullanıcı arayüzünü göstermektedir. Kalite boyutları, radar grafik ile kalite özeti ve kritiklik dağılımları görselleştirilmektedir.



Şekil 5. Analiz sayfası kalite metriklerinin görünümü.

5. Deneysel Değerlendirme

5.1. Deney Ortamı

Önerilen yaklaşımın başarımı, üç farklı uygulama senaryosu kapsamında ampirik olarak değerlendirilmiştir. Kullanılan veri setleri, ilgili alanların özgün gereksinimleri ve etki alanına özgü parametreler doğrultusunda üretilen sentetik verilerden oluşmakta; bileşen sayısı, topolojik yoğunluk ve Hizmet Kalitesi (QoS) yapılandırılmaları açısından gerçek dünya sistemlerinin ölçeğini yansıtmaktadır. Deneysel çalışmalar, Intel Core i7-12700 işlemci, 32 GB bellek ve Ubuntu 22.04 işletim sistemine sahip bir hesaplama altyapısı üzerinde yürütülmüştür. ROS 2 [14] otonom sistemler kapsamında 15 düğüm, 3 aracı, 45 uygulama ve 120 konu içeren bir otonom araç yazılım sistemi kullanılmıştır. IoT Akıllı Şehir senaryosunda 50 düğüm, 5 aracı, 200 uygulama ve 500 konu içeren bir akıllı şehir sensör ağı incelenmiştir. Finansal İşlem Platformu olarak Kafka [15] tabanlı 10 düğüm, 8 aracı, 80 uygulama ve 300 konu içeren bir yüksek frekanslı işlem sistemi değerlendirilmiştir.

5.2. Doğrulama Sonuçları

Kritiklik tahminlerinin doğruluğu, simülasyon tabanlı etki analizi ile karşılaştırılmıştır. Her bileşen için hata simülasyonu gerçekleştirilmiş ve gerçek etki puanı $I(v)$ hesaplanmıştır. Topolojik metriklerden türetilen kalite puanı $Q(v)$ ile simülasyon sonucu etki puanı arasındaki korelasyon ve örtüşme ölçülmüştür. Tablo 1'de doğrulama metrikleri sunulmaktadır.

Tablo 1. Doğrulama Metrikleri

Metrik	ROS2 Otonom Sistem	IoT Akıllı Şehir	Kafka Finans Platformu
Spearman ρ	0.865	0.890	0.900
F1 Skor	0.750	0.840	0.867
Kesinlik	0.750	0.840	0.867
Duyarlılık	0.750	0.840	0.867
İlk-5 Örtüşme	%40	%100	%80

Tablo 1 verileri tetkik edildiğinde, topolojik olarak hesaplanan $Q(v)$ kalite puanı ile simülasyon temelli gerçek etki puanı $I(v)$ arasında tüm uygulama senaryoları özelinde kuvvetli bir korelasyonun varlığı müşahade edilmektedir. Spearman ρ katsayılarının 0,865 ile 0,900 aralığında seyretmesi, önerilen topoloji odaklı metodolojinin çalışma zamanı izleme maliyetlerine gereksinim duymaksızın bileşenlerin etki sıralamasını yüksek doğruluk payı ile öngörebildiğini kanıtlamaktadır. Azami F1 skoru ve %100 düzeyindeki İlk-5 örtüşme oranına IoT Akıllı Şehir senaryosunda ulaşılmış olup, bu durum söz konusu senaryonun nispeten homojen nitelikteki konu–uygulama mimarisinin topolojik sinyal yoğunluğunu artırmasına atfedilmektedir. Kafka tabanlı Finansal İşlem Platformu en yüksek Spearman korelasyon değerine (0,900) erişmiş olsa da, F1 skoru bakımından IoT senaryosunun bir miktar gerisinde kalmıştır; bu olgu, aracı yoğunluklu Kafka topolojisinin sıralama korelasyonu için uygun bir zemin hazırlarken, kritiklik sınırlarının belirlenmesinde belirsizlikler yaratabildiğini göstermektedir. ROS 2 Otonom Sistemler senaryosunda ise F1 skoru ve İlk-5 örtüşme oranları, diğer iki kullanım durumuna kıyasla daha düşük seviyelerde gerçekleşmiştir. Bu bulgu, otonom araç yazılım ekosistemlerinde görev ve güvenlik kritik bileşenlerinin operasyonel hiyerarşisinin, yapısal bağımlılık örüntülerinden ziyade işlevsel önceliklere dayanması nedeniyle topolojik kestirimin daha kompleks bir mahiyet arz ettiğini ortaya koymaktadır. İncelenen tüm senaryolarda kesinlik ve duyarlılık değerleri arasında gözlemlenen tutarlılık, kutu grafiği tabanlı istatistiksel sınıflandırma yaklaşımının kritik olan ve olmayan bileşenlerin ayrıştırılmasında dengeli bir eşikleme mekanizması sunduğunu doğrulamaktadır.

5.3. Performans Değerlendirmesi

Önerilen görselleştirme yaklaşımı, 1000'den fazla bileşen içeren sistemlere verimli şekilde ölçeklenmektedir. Web platformu için ölçeklenebilirlik testleri, 10.000'den fazla bileşen içeren sistemlerde bile görselleştirmenin kabul edilebilir sürelerde tamamlandığını göstermiştir. WebGL tabanlı kuvvet yönlendirmeli çizge görüntülemesinde 60 FPS akıcılık sağlanmış, 3D modda 5.000 düğümlü grafiklerde etkileşimli gezinme mümkün olmuştur. Tablo 2'de performans ölçümleri verilmektedir.

Tablo 2. Performans Ölçümleri

Bileşen	Bağımlılık	Yükleme	Analiz	Görüntüleme (2D/3D)
25	69	0,28 s	0,02 s	1,73 / 1,39 s
100	335	0,48 s	0,05 s	1,68 / 1,62 s
500	1.437	1,66 s	0,32 s	2,25 / 3,64 s
1.000	4.267	4,57 s	4,60 s	2,65 / 6,50 s
10.000	11.313	104,50 s	11,02 s	4,30 / 6,38 s

6. Sonuç

Bu çalışmada, yayınlı-abone ol tabanlı dağıtık sistemlerin çizge modeli ile görselleştirilmesine yönelik kapsamlı bir web platformu sunulmuştur. Platform, sistem topolojisini formal çizge yapısına dönüştürerek topolojik metrikler ve kutu grafiği tabanlı istatistiksel sınıflandırma aracılığıyla kritik bileşenlerin tespit edilmesini sağlamaktadır. Next.js ve React tabanlı modern web arayüzü, 2D/3D kuvvet yönlendirmeli çizge görselleştirmesi, FastAPI backend servisleri ve Neo4j çizge veritabanı entegrasyonu ile kurumsal ölçekte kullanılabilir bir çözüm geliştirilmiştir.

Üç farklı etki alanında gerçekleştirilen deneysel değerlendirme, önerilen yaklaşımın tüm senaryolarda güçlü topolojik tahmin kabiliyeti sergilediğini ortaya koymaktadır. Spearman korelasyon katsayıları 0,865 ile 0,900 aralığında seyretmiş; F1 skoru ise 0,750 ile 0,867 arasında gerçekleşmiştir. En yüksek başarımlı IoT Akıllı Şehir senaryosunda elde edilirken, ROS 2 tabanlı otonom sistem senaryosu görev-güvenlik odaklı bileşen önceliklendirmesi nedeniyle daha düşük İlk-5 örtüşme oranı sergilemiştir. Bu bulgu, topoloji temelli yaklaşımın genel geçerliliğini doğrularken, güvenlik kritik gömülü sistemler gibi alana özgü bağlamlarda ek etki alanı sinyallerinin modele entegre edilmesinin faydalı olabileceğine işaret etmektedir.

Gelecek çalışmalarda, çizge sinir ağları (Graph Neural Networks) kullanılarak kritiklik puanlamasının iyileştirilmesi, zamansal çizge evrimi analizi ve alana özgü bağlam bilgisinin modele dahil edilmesi planlanmaktadır. Ayrıca metodolojinin REST API, GraphQL ve mikroservis mimarileri gibi diğer mimari stillere genişletilmesi ile gerçek üretim ortamlarından elde edilecek anonim veri setleri üzerinde doğrulama çalışmalarının yürütülmesi hedeflenmektedir.

Teşekkür

Bu çalışma, HAVELSAN A.Ş. ve İstanbul Teknik Üniversitesi Bilgisayar Mühendisliği Bölümü işbirliği ile yürütülmüştür.

Üretken Yapay Zekâ Kullanım Beyanı

Bu çalışmanın hazırlanması sürecinde yazar(lar) dilbilgisi ve yazım kontrolü amacıyla yapay zekâ araçlarından yararlanmıştır. Bu araç(lar)/hizmet(ler) kullanıldıktan sonra, yazar(lar) içeriği gerektiği şekilde gözden geçirip düzenlemiş ve yayının içeriği için tam sorumluluğu almaktadır.

Kaynaklar

- [1] Eugster, P.T., Felber, P.A., Guerraoui, R., Kermarrec, A.M.: The many faces of publish/subscribe. ACM Computing Surveys 35(2), 114-131 (2003)
- [2] Yiğit, İ. O., Buzluca, F.: A Graph-Based Dependency Analysis Method for Identifying Critical Components in Distributed Publish-Subscribe Systems. In: 2025 IEEE International Conference on Recent Advances in Systems Science and Engineering (RASSE), Singapore, pp. 1-9 (2025). doi: 10.1109/RASSE64831.2025.11315354
- [3] Sambasivan, R.R., Zheng, A.X., De Rosa, M., Krevat, E., Whitman, S., Stroucken, M., Wang, W., Xu, L., Ganger, G.R.: Diagnosing performance changes by comparing request flows. In: NSDI, pp. 43-56 (2011)
- [4] Bastian, M., Heymann, S., Jacomy, M.: Gephi: An open source software for exploring and manipulating networks. In: ICWSM, pp. 361-362 (2009)
- [5] Ellson, J., Gansner, E., Koutsofios, L., North, S.C., Woodhull, G.: Graphviz—open source graph drawing tools. In: International Symposium on Graph Drawing, pp. 483-484 (2001)
- [6] Zimmermann, T., Nagappan, N.: Predicting defects using network analysis on dependency graphs. In: ICSE, pp. 531-540 (2008)
- [7] Cai, Y., Kazman, R., Jaspan, C., Herbsleb, J.: Architectural decay in open-source software: A temporal study. In: ICSA, pp. 101-110 (2019)

- [8] Page, L., Brin, S., Motwani, R., Winograd, T.: The PageRank citation ranking: Bringing order to the web. Technical Report, Stanford InfoLab (1999)
- [9] Freeman, L.C.: A set of measures of centrality based on betweenness. *Sociometry* 40(1), 35-41 (1977)
- [10] Kleinberg, J.M.: Authoritative sources in a hyperlinked environment. *Journal of the ACM* 46(5), 604-632 (1999)
- [11] Vasco Asturiano: react-force-graph - React component for 2D/3D force-directed graphs. <https://github.com/vasturiano/react-force-graph> (2024)
- [12] Neo4j, Inc.: Neo4j Graph Database Platform. <https://neo4j.com> (2024)
- [13] Hagberg, A., Swart, P., Chult, D.S.: Exploring network structure, dynamics, and function using NetworkX. In: *SciPy*, pp. 11-15 (2008)
- [14] Macenski, S., Foote, T., Gerkey, B., Lalancette, C., Woodall, W.: Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics* 7(66), eabm6074 (2022)
- [15] Kreps, J., Narkhede, N., Rao, J.: Kafka: A distributed messaging system for log processing. In: *NetDB Workshop*, pp. 1-7 (2011)
- [16] Saaty, T.L.: *The analytic hierarchy process: Planning, priority setting, resource allocation*. McGraw-Hill, New York (1980)
- [17] Oskarsson, J., Hu, F.: Distributed systems tracing with Zipkin. *Twitter Engineering Blog*. https://blog.twitter.com/engineering/en_us/a/2012/distributed-systems-tracing-with-zipkin (2012)
- [18] The Jaeger Authors: Jaeger: open source, end-to-end distributed tracing. Cloud Native Computing Foundation. <https://www.jaegertracing.io> (2016)